

Binary Bloom Vault

Complete Technical Specification & Source Code Documentation

System Architecture Report

Generated on: 2026

Language Environment: Python 3.10+

1. Architectural Overview

The **Binary Bloom Vault** system is designed as a standalone secure text repository that blends private personal notes management, text optimization, data-trail logs, and multi-user collaboration patterns. Written in native Python, it requires zero external dependencies, making it clean, portable, and optimal for isolated execution environments like custom command-line interfaces or terminal-focused shells.

Core Operating Philosophy

The system prioritizes local structural integrity, utilizing atomic JSON-based state saves and modern cryptographic mechanisms to securely store identities, posts, text characteristics, and chronological tracking branches.

2. Detailed Code Breakdown

This section breaks down the application logic module-by-module, analyzing how data flows from user input down into persistent storage frames.

2.1 Terminal Rendering & Visual Wrapping

Visual representation is kept isolated inside the `C` class using standardized 16-color ANSI escape sequences. Rather than polluting normal terminal print declarations, the `color(text, code)` utility wraps individual payloads dynamically. The `banner()` wrapper orchestrates complex multi-color headers cleanly, signaling secure entry states upon execution initialization.

2.2 Cryptographic Security Layer

Authentication operations discard classical plaintext mapping. Instead, accounts rely on high-entropy mechanisms:

- **Dynamic Salt Generation:** Every registered user receives a unique, random hexadecimal sequence built via `secrets.token_hex(16)`. This defeats precomputed rainbow table attacks entirely.
- **PBKDF2 Iterative Hashing:** Passwords undergo stretching via the `hashlib.pbkdf2_hmac` system, configured to utilize SHA-256 for 120,000 computation cycles. This significantly limits brute-force viability.
- **Constant-Time Verification:** Validating incoming validation metrics avoids baseline string matching. It utilizes `secrets.compare_digest` to effectively shield the ecosystem from structural side-channel timing profiling.

2.3 Data Analysis & Translation Components

The repository features integrated analytic parsers that operate asynchronously every time a state changes:

- `analyze_text(text)` splits text elements, automatically strips down punctuation arrays, cleans string casing, handles lengths, and calculates word metrics.
- `text_to_binary(text)` converts standard strings down into pure structural byte strings using 8-bit spacing blocks, creating a unique data fingerprint for storage verification.

2.4 Dataclass Schema Models

The entity layer models data cleanly by leveraging native Python `@dataclass` primitives, establishing reliable, type-hinted database tables:

- `BranchSuggestion` manages proposals with explicit statuses (*pending*, *accepted*, *rejected*).
- `Post` encapsulates core payload contents, analysis tables, binaries, structural branch pointers, and internal event loops.
- `UserAccount` bridges security arrays and structures collections mapping independent entries sequentially to individual workspace schemas.

2.5 The Persistent State Engine

The application engine is encapsulated inside the `DepositorySystem` controller class. It functions both as an transactional router and an input/output layer:

- **Storage Layer:** The `save()` routine transforms structural model collections into generic native primitives via nested `asdict()` serialization loops, writing atomically directly to `depository_data.json`.
- **Hydration Layer:** On startup, `load()` evaluates file existence, unrolls the raw JSON payload, and fully instantiates the nested class architecture to populate the running application memory.
- **Branch & Git-style Workspace Workflow:** Collaborative content updates do not immediately impact real data. Proposals exist inside an isolated branch pool until owners explicitly accept or reject changes, mapping old versions out to explicit historical transaction logs.

3. Full Implementation Source Code

The production codebase for the complete engine is appended below in its entirety. It includes all terminal styling frameworks, structural helper logic, dataclass collections, persistent databases, transactional interfaces, and main execution loops.

```
import json
import os
import hashlib
import secrets
from dataclasses import dataclass, asdict, field
from datetime import datetime
from typing import Dict, List, Optional

# =====
# Simple built-in terminal colors using ANSI escape codes
# =====
class C:
    RESET = "\033[0m"
    BOLD = "\033[1m"
    DIM = "\033[2m"
    RED = "\033[31m"
    GREEN = "\033[32m"
```

```

YELLOW = "\033[33m"
BLUE = "\033[34m"
MAGENTA = "\033[35m"
CYAN = "\033[36m"
WHITE = "\033[37m"

def color(text: str, code: str) -> str:
    return f"{code}{text}{C.RESET}"

def banner() -> None:
    print("\n" + color("=" * 66, C.CYAN))
    print(color("                BINARY BLOOM VAULT", C.BOLD + C.MAGENTA))
    print(color("        Private vaults, public posts, binary trails, branches", C.CYAN))
    print(color("=" * 66, C.CYAN))

# =====
# Helper functions
# =====

def current_time() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

def prompt_password(prompt: str) -> str:
    """
    Visible password input on purpose.
    Some terminals hide input weirdly with getpass, so we keep it simple.
    """
    return input(prompt)

def text_to_binary(text: str) -> str:
    return " ".join(format(ord(ch), "08b") for ch in text)

def binary_summary(text: str) -> str:
    bits = text_to_binary(text).replace(" ", "")
    return bits[:80] + ("..." if len(bits) > 80 else "")

def analyze_text(text: str) -> dict:
    words = [w.strip(".!?:\n'()[]{}<>-").lower() for w in text.split() if w.strip()]
    return {
        "word_count": len(words),
        "char_count": len(text),
        "unique_words": len(set(words)),
        "longest_word": max(words, key=len) if words else "",
        "sentence_count": max(1, text.count(".") + text.count("!") + text.count("?"))
    }
    if text.strip() else 0,
    }

```

```
def hash_password(password: str, salt: Optional[str] = None) -> tuple[str, str]:
    if salt is None:
        salt = secrets.token_hex(16)
    hashed = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt.encode("utf-8"),
        120000,
    ).hex()
    return salt, hashed
```

```
def verify_password(password: str, salt: str, stored_hash: str) -> bool:
    _, test_hash = hash_password(password, salt)
    return secrets.compare_digest(test_hash, stored_hash)
```

```
# =====
# Data models
# =====
```

```
@dataclass
class BranchSuggestion:
    branch_id: int
    from_user: str
    suggestion: str
    note: str
    created_at: str
    status: str = "pending" # pending / accepted / rejected
```

```
@dataclass
class Post:
    post_id: int
    title: str
    content: str
    tags: List[str]
    owner: str
    created_at: str
    analysis: dict = field(default_factory=dict)
    binary_version: str = ""
    history: List[dict] = field(default_factory=list)
    branches: List[BranchSuggestion] = field(default_factory=list)

    def refresh(self) -> None:
        self.analysis = analyze_text(self.content)
        self.binary_version = text_to_binary(self.content)
```

```
@dataclass
class UserAccount:
    username: str
    salt: str
```

```

password_hash: str
created_at: str
posts: Dict[int, Post] = field(default_factory=dict)

# =====
# Core system
# =====

class DepositorySystem:
    def __init__(self, filename: str = "depository_data.json"):
        self.filename = filename
        self.users: Dict[str, UserAccount] = {}
        self.load()

    def register_user(self, username: str, password: str) -> tuple[bool, str]:
        username = username.strip().lower()

        if not username:
            return False, "Username cannot be empty."
        if username in self.users:
            return False, "That username already exists."
        if len(password) < 6:
            return False, "Password too short. Use at least 6 characters."

        salt, pwd_hash = hash_password(password)
        self.users[username] = UserAccount(
            username=username,
            salt=salt,
            password_hash=pwd_hash,
            created_at=current_time(),
        )
        self.save()
        return True, "Account created."

    def login_user(self, username: str, password: str) -> tuple[bool, str]:
        username = username.strip().lower()
        if username not in self.users:
            return False, "Username not found."

        account = self.users[username]
        if not verify_password(password, account.salt, account.password_hash):
            return False, "Wrong password. Access denied."

        return True, "Login successful."

    def next_post_id(self, username: str) -> int:
        account = self.users[username]
        return max(account.posts.keys(), default=0) + 1

    def next_branch_id(self) -> int:
        highest = 0
        for account in self.users.values():
            for post in account.posts.values():

```

```

        for branch in post.branches:
            highest = max(highest, branch.branch_id)
        return highest + 1

def create_post(self, username: str, title: str, content: str, tags: List[str]) ->
Post:
    account = self.users[username]
    post = Post(
        post_id=self.next_post_id(username),
        title=title.strip(),
        content=content.strip(),
        tags=[t.strip().lower() for t in tags if t.strip()],
        owner=username,
        created_at=current_time(),
    )
    post.refresh()
    post.history.append({
        "event": "created",
        "time": post.created_at,
        "content": post.content,
        "binary": post.binary_version,
    })
    account.posts[post.post_id] = post
    self.save()
    return post

def update_post_content(self, username: str, post_id: int, new_content: str) ->
bool:
    account = self.users[username]
    if post_id not in account.posts:
        return False

    post = account.posts[post_id]
    old_content = post.content
    post.content = new_content.strip()
    post.refresh()

    post.history.append({
        "event": "owner_edit",
        "time": current_time(),
        "old_content": old_content,
        "new_content": post.content,
        "old_binary": text_to_binary(old_content),
        "new_binary": post.binary_version,
    })
    self.save()
    return True

def add_branch(self, owner: str, post_id: int, from_user: str, suggestion: str,
note: str) -> bool:
    owner = owner.strip().lower()
    if owner not in self.users:
        return False
    account = self.users[owner]

```



```

        if post_id not in account.posts:
            return False

        post = account.posts[post_id]
        branch = BranchSuggestion(
            branch_id=self.next_branch_id(),
            from_user=from_user.strip().lower(),
            suggestion=suggestion.strip(),
            note=note.strip(),
            created_at=current_time(),
        )
        post.branches.append(branch)
        post.history.append({
            "event": "branch_added",
            "time": branch.created_at,
            "from_user": branch.from_user,
            "note": branch.note,
        })
        self.save()
        return True

def apply_branch(self, owner: str, post_id: int, branch_id: int) -> bool:
    owner = owner.strip().lower()
    if owner not in self.users:
        return False
    account = self.users[owner]
    if post_id not in account.posts:
        return False

    post = account.posts[post_id]
    for branch in post.branches:
        if branch.branch_id == branch_id and branch.status == "pending":
            old_content = post.content
            post.content = branch.suggestion
            post.refresh()
            branch.status = "accepted"
            post.history.append({
                "event": "branch_accepted",
                "time": current_time(),
                "from_user": branch.from_user,
                "old_content": old_content,
                "new_content": post.content,
                "old_binary": text_to_binary(old_content),
                "new_binary": post.binary_version,
            })
            self.save()
            return True
    return False

def reject_branch(self, owner: str, post_id: int, branch_id: int) -> bool:
    owner = owner.strip().lower()
    if owner not in self.users:
        return False
    account = self.users[owner]

```

```

    if post_id not in account.posts:
        return False

    post = account.posts[post_id]
    for branch in post.branches:
        if branch.branch_id == branch_id and branch.status == "pending":
            branch.status = "rejected"
            post.history.append({
                "event": "branch_rejected",
                "time": current_time(),
                "from_user": branch.from_user,
                "branch_id": branch.branch_id,
            })
            self.save()
            return True
    return False

def list_user_posts(self, username: str) -> List[Post]:
    username = username.strip().lower()
    if username not in self.users:
        return []
    return sorted(self.users[username].posts.values(), key=lambda p: p.post_id)

def get_post(self, owner: str, post_id: int) -> Optional[Post]:
    owner = owner.strip().lower()
    if owner not in self.users:
        return None
    return self.users[owner].posts.get(post_id)

def public_feed(self) -> List[Post]:
    all_posts = []
    for account in self.users.values():
        all_posts.extend(account.posts.values())
    return sorted(all_posts, key=lambda p: (p.created_at, p.post_id),
reverse=True)

def save(self) -> None:
    data = {
        "users": {
            username: {
                "username": account.username,
                "salt": account.salt,
                "password_hash": account.password_hash,
                "created_at": account.created_at,
                "posts": {
                    str(pid): {
                        **asdict(post),
                        "branches": [asdict(b) for b in post.branches],
                    }
                    for pid, post in account.posts.items()
                },
            }
            for username, account in self.users.items()
        }
    }

```

```

    }
    with open(self.filename, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)

def load(self) -> None:
    if not os.path.exists(self.filename):
        return

    with open(self.filename, "r", encoding="utf-8") as f:
        data = json.load(f)

    for username, account_data in data.get("users", {}).items():
        account = UserAccount(
            username=account_data["username"],
            salt=account_data["salt"],
            password_hash=account_data["password_hash"],
            created_at=account_data["created_at"],
            posts={},
        )
        for _, post_data in account_data.get("posts", {}).items():
            branches = [BranchSuggestion(**b) for b in post_data.get("branches",
[[]])

                post = Post(
                    post_id=post_data["post_id"],
                    title=post_data["title"],
                    content=post_data["content"],
                    tags=post_data["tags"],
                    owner=post_data["owner"],
                    created_at=post_data["created_at"],
                    analysis=post_data.get("analysis", {}),
                    binary_version=post_data.get("binary_version", ""),
                    history=post_data.get("history", []),
                    branches=branches,
                )
                account.posts[post.post_id] = post
        self.users[username] = account

# =====
# Display helpers
# =====

def show_post(post: Post) -> None:
    print(color("-" * 72, C.BLUE))
    print(color(f"Post ID: {post.post_id}", C.BOLD + C.CYAN))
    print(color(f"Title : {post.title}", C.WHITE))
    print(color(f"Owner : {post.owner}", C.WHITE))
    print(color(f"Tags : {' '.join(post.tags) if post.tags else 'None'}", C.WHITE))
    print(color(f"Time : {post.created_at}", C.DIM + C.WHITE))
    print(color(f"Content: {post.content}", C.YELLOW))
    print(color(f"Binary : {binary_summary(post.content)}", C.MAGENTA))
    print(color(
        f"Analysis -> words={post.analysis.get('word_count', 0)}, "
        f"chars={post.analysis.get('char_count', 0)}, "

```

```

        f"unique={post.analysis.get('unique_words', 0)}, "
        f"longest='{post.analysis.get('longest_word', '')}'\"",
        C.GREEN,
    ))
    print(color(f"Branches: {len(post.branches)}", C.CYAN))
    print(color(f"History : {len(post.history)} event(s)", C.CYAN))
    print(color("-" * 72, C.BLUE))

def show_branches(post: Post) -> None:
    if not post.branches:
        print(color("No branches yet.", C.DIM + C.WHITE))
        return
    print(color("Branch suggestions:", C.BOLD + C.MAGENTA))
    for b in post.branches:
        print(color(
            f" [{b.branch_id}] from {b.from_user} | {b.status} | note: {b.note}",
            C.WHITE,
        ))
        print(color(f"          suggestion: {b.suggestion}", C.YELLOW))

# =====
# User flows
# =====

def register_flow(system: DepositorySystem) -> Optional[str]:
    banner()
    print(color("Create your vault account.", C.BOLD + C.GREEN))
    print(color("WARNING: passwords are saved as a one-way hash.", C.YELLOW))
    print(color("If you forget the password, it cannot be recovered.", C.RED))
    print()

    username = input(color("Choose a username: ", C.CYAN)).strip().lower()
    password = prompt_password(color("Choose a password: ", C.CYAN))
    confirm = prompt_password(color("Confirm password: ", C.CYAN))

    if password != confirm:
        print(color("Password mismatch. Account not created.", C.RED))
        return None

    ok, message = system.register_user(username, password)
    if not ok:
        print(color(message, C.RED))
        return None

    print(color("Account created successfully.", C.GREEN))
    print(color("Password stored securely as a hash.", C.YELLOW))
    return username

def login_flow(system: DepositorySystem) -> Optional[str]:
    banner()
    print(color("Login to your vault.", C.BOLD + C.GREEN))

```

```

print(color("Wrong password warning: access will be denied.", C.YELLOW))
print()

username = input(color("Username: ", C.CYAN)).strip().lower()
password = prompt_password(color("Password: ", C.CYAN))

ok, message = system.login_user(username, password)
if not ok:
    print(color(message, C.RED))
    return None

print(color(f"Welcome back, {username}.", C.GREEN))
return username

def account_menu(system: DepositorySystem, username: str) -> None:
    while True:
        banner()
        print(color(f"Logged in as: {username}", C.BOLD + C.WHITE))
        print(color("1.", C.CYAN), "Create post")
        print(color("2.", C.CYAN), "View my posts")
        print(color("3.", C.CYAN), "Edit my post")
        print(color("4.", C.CYAN), "Public feed (everyone can see)")
        print(color("5.", C.CYAN), "Suggest branch to another user's post")
        print(color("6.", C.CYAN), "Review branches on my post")
        print(color("7.", C.CYAN), "View one post history")
        print(color("8.", C.CYAN), "Save vault")
        print(color("9.", C.CYAN), "Log out")
        print(color("0.", C.CYAN), "Exit")

        choice = input(color("\nChoose: ", C.MAGENTA)).strip()

        if choice == "1":
            title = input(color("Title: ", C.CYAN))
            content = input(color("Content: ", C.CYAN))
            tags = input(color("Tags (comma separated): ", C.CYAN)).split(",")
            post = system.create_post(username, title, content, tags)
            print(color("Post created.", C.GREEN))
            print(color(f"Instant analysis: {post.analysis}", C.YELLOW))
            print(color(f"Binary trail: {binary_summary(post.content)}", C.MAGENTA))

        elif choice == "2":
            posts = system.list_user_posts(username)
            if not posts:
                print(color("No posts in your vault yet.", C.YELLOW))
            else:
                for post in posts:
                    show_post(post)

        elif choice == "3":
            try:
                post_id = int(input(color("Post ID to edit: ", C.CYAN)))
                new_content = input(color("New content: ", C.CYAN))
                if system.update_post_content(username, post_id, new_content):

```

```

        print(color("Post updated.", C.GREEN))
    else:
        print(color("Post not found.", C.RED))
except ValueError:
    print(color("Enter a valid number.", C.RED))

elif choice == "4":
    feed = system.public_feed()
    if not feed:
        print(color("No public posts yet.", C.YELLOW))
    else:
        print(color("PUBLIC FEED", C.BOLD + C.MAGENTA))
        for post in feed:
            show_post(post)

elif choice == "5":
    owner = input(color("Owner username: ", C.CYAN)).strip().lower()
    try:
        post_id = int(input(color("Post ID: ", C.CYAN)))
        suggestion = input(color("Your suggested version: ", C.CYAN))
        note = input(color("Short note: ", C.CYAN))
        ok = system.add_branch(owner, post_id, username, suggestion, note)
        if ok:
            print(color("Branch suggestion added.", C.GREEN))
        else:
            print(color("Post not found.", C.RED))
    except ValueError:
        print(color("Enter a valid number.", C.RED))

elif choice == "6":
    try:
        post_id = int(input(color("Your post ID: ", C.CYAN)))
        post = system.get_post(username, post_id)
        if not post:
            print(color("Post not found.", C.RED))
            continue

        show_post(post)
        show_branches(post)

        action = input(color("Type 'a' to accept, 'r' to reject, or Enter to
cancel: ", C.CYAN)).strip().lower()
        if action in {"a", "r"}:
            try:
                branch_id = int(input(color("Branch ID: ", C.CYAN)))
                if action == "a":
                    ok = system.apply_branch(username, post_id, branch_id)
                    print(color("Branch accepted.", C.GREEN) if ok else
color("Could not accept branch.", C.RED))
                else:
                    ok = system.reject_branch(username, post_id, branch_id)
                    print(color("Branch rejected.", C.GREEN) if ok else
color("Could not reject branch.", C.RED))
            except ValueError:

```

```

        print(color("Enter a valid number.", C.RED))
    except ValueError:
        print(color("Enter a valid number.", C.RED))

elif choice == "7":
    try:
        post_id = int(input(color("Your post ID: ", C.CYAN)))
        post = system.get_post(username, post_id)
        if not post:
            print(color("Post not found.", C.RED))
        else:
            print(color("HISTORY LOG", C.BOLD + C.MAGENTA))
            for item in post.history:
                print(color(str(item), C.WHITE))
    except ValueError:
        print(color("Enter a valid number.", C.RED))

elif choice == "8":
    system.save()
    print(color("Vault saved.", C.GREEN))
    print(color("Reminder: passwords are hashed and cannot be recovered.",
C.YELLOW))

elif choice == "9":
    system.save()
    print(color("Logged out.", C.GREEN))
    break

elif choice == "0":
    system.save()
    print(color("Saved and exiting.", C.GREEN))
    raise SystemExit

else:
    print(color("Invalid choice.", C.RED))

def main():
    system = DepositorySystem()

    while True:
        banner()
        print(color("1.", C.CYAN), "Register")
        print(color("2.", C.CYAN), "Login")
        print(color("0.", C.CYAN), "Exit")

        choice = input(color("\nChoose: ", C.MAGENTA)).strip()

        if choice == "1":
            username = register_flow(system)
            if username:
                account_menu(system, username)

        elif choice == "2":

```

```
        username = login_flow(system)
        if username:
            account_menu(system, username)

    elif choice == "0":
        system.save()
        print(color("Goodbye.", C.GREEN))
        break

    else:
        print(color("Invalid choice.", C.RED))

if __name__ == "__main__":
    main()
```